

2025-04-25

Contents

0.1	EDA	1
0.2		1
0.3		2
0.4		3
0.5		4

0.1 EDA

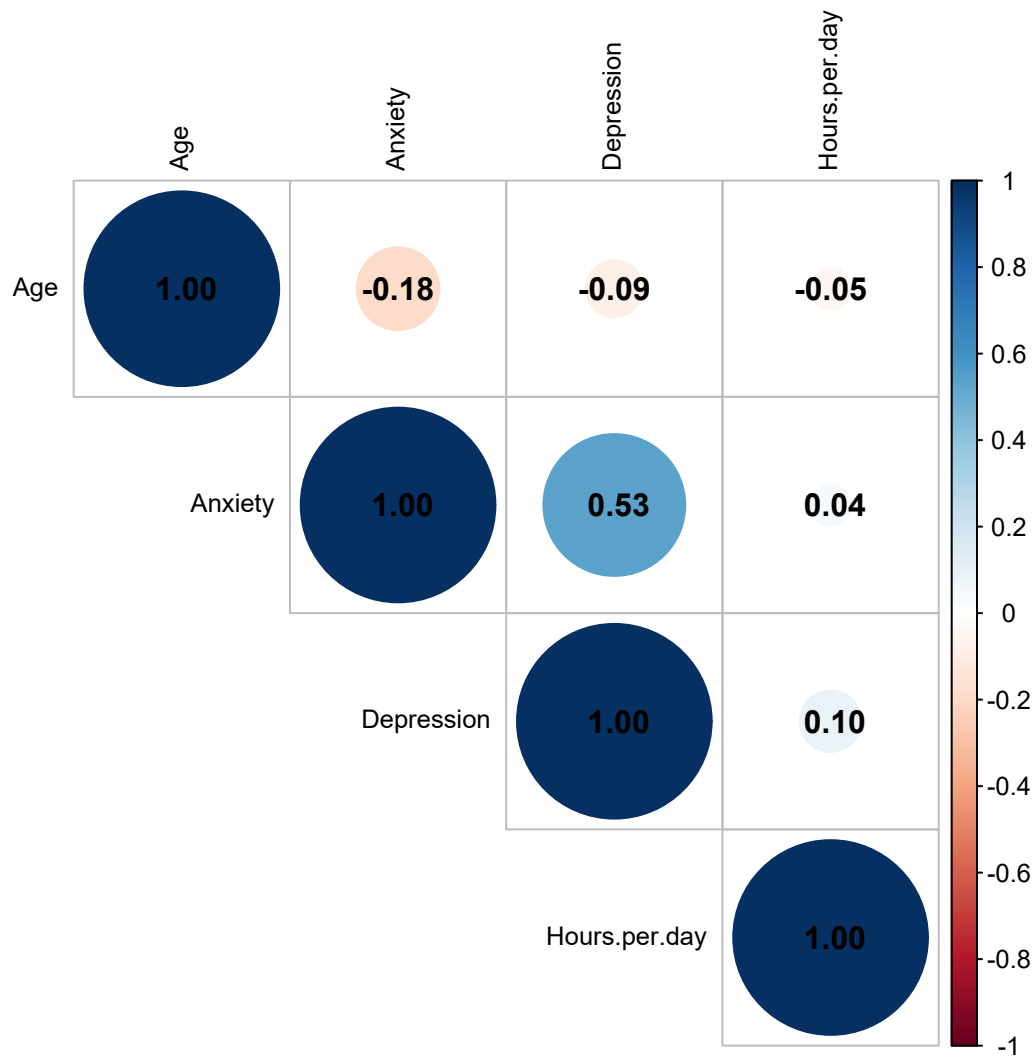
```
library(corrplot)
df <- read.csv("mxmh_survey_results.csv")
df <- na.omit(df)

#
summary(df[, c("Age", "Anxiety", "Depression", "Hours.per.day")])
```

##	Age	Anxiety	Depression	Hours.per.day
##	Min. :10.00	Min. : 0.000	Min. : 0.000	Min. : 0.000
##	1st Qu.:18.00	1st Qu.: 4.000	1st Qu.: 2.000	1st Qu.: 2.000
##	Median :21.00	Median : 6.000	Median : 5.000	Median : 3.000
##	Mean :24.76	Mean : 5.874	Mean : 4.887	Mean : 3.711
##	3rd Qu.:27.00	3rd Qu.: 8.000	3rd Qu.: 7.000	3rd Qu.: 5.000
##	Max. :89.00	Max. :10.000	Max. :10.000	Max. :24.000

0.2

```
#
corr_matrix <- cor(df[, c("Age", "Anxiety", "Depression", "Hours.per.day")])
#
corrplot(corr_matrix, method = "circle", type = "upper",
         addCoef.col = "black", tl.col = "black", tl.cex = 0.8)
```



0.3

```
#
model_lm <- lm(Anxiety ~ Depression + Hours.per.day + Age, data = df)

#
summary(model_lm)

##
## Call:
## lm(formula = Anxiety ~ Depression + Hours.per.day + Age, data = df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -7.7547 -1.4956  0.1875  1.4624  6.1696
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   4.379012   0.293613   14.914 < 2e-16 ***
## Depression     0.478968   0.031064   15.419 < 2e-16 ***
```

```
## Hours.per.day -0.012752  0.030160 -0.423    0.673
## Age          -0.032270  0.007959 -4.055 5.65e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.32 on 625 degrees of freedom
## Multiple R-squared:  0.3, Adjusted R-squared:  0.2966
## F-statistic: 89.28 on 3 and 625 DF, p-value: < 2.2e-16
```

0.4

```
library(caret)
library(nnet)
#           Timestamp
if ("Timestamp" %in% colnames(df)) {
  df$Timestamp <- NULL
}

#
cat_columns <- names(df)[sapply(df, is.character)]
for (col in cat_columns) {
  #
  df[[col]] <- factor(df[[col]])
  #
  df[[col]] <- as.numeric(df[[col]])
}

#
numeric_columns <- names(df)[sapply(df, is.numeric)]
for (col in numeric_columns) {
  df[[col]][is.na(df[[col]])] <- mean(df[[col]], na.rm = TRUE)
}

#
df$Music.effects <- as.factor(df$Music.effects)

#
X_class <- df[, !names(df) %in% c("Music.effects", "Anxiety", "Depression")]
y_class <- df$Music.effects
temp_class <- cbind(X_class, Music.effects = y_class)
set.seed(42)
trainIndex_class <- createDataPartition(temp_class$Music.effects, p = 0.7, list = FALSE)
train_class <- temp_class[trainIndex_class, ]
test_class <- temp_class[-trainIndex_class, ]
model <- multinom(Music.effects ~ ., data = train_class)

## # weights:  124 (90 variable)

## initial  value 612.742108
## iter   10 value 610.908231
## iter   20 value 608.529722
## iter   30 value 604.938124
## iter   40 value 598.614645
## iter   50 value 594.856931
```

```

## iter 60 value 589.928399
## iter 70 value 578.346700
## iter 80 value 569.341923
## iter 90 value 563.681111
## iter 100 value 560.919586
## final value 560.919586
## stopped after 100 iterations

#
pred_class <- predict(model, newdata = test_class)

#
levels_test <- levels(test_class$Music.effects)
pred_class <- factor(pred_class, levels = levels_test)
actual_class <- factor(test_class$Music.effects, levels = levels_test)

conf_matrix <- confusionMatrix(pred_class, actual_class)
print(conf_matrix)

## Confusion Matrix and Statistics
##
##           Reference
## Prediction  1    2    3    4
##           1    0    0    0    0
##           2   1 138   38    3
##           3    0    3    3    1
##           4    0    0    0    0
##
## Overall Statistics
##
##           Accuracy : 0.754
##           95% CI : (0.6859, 0.8139)
##           No Information Rate : 0.754
##           P-Value [Acc > NIR] : 0.5395
##
##           Kappa : 0.0753
##
## McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: 1 Class: 2 Class: 3 Class: 4
## Sensitivity      0.000000  0.97872  0.07317  0.00000
## Specificity      1.000000  0.08696  0.97260  1.00000
## Pos Pred Value    NaN  0.76667  0.42857    NaN
## Neg Pred Value    0.994652  0.57143  0.78889  0.97861
## Prevalence        0.005348  0.75401  0.21925  0.02139
## Detection Rate    0.000000  0.73797  0.01604  0.00000
## Detection Prevalence 0.000000  0.96257  0.03743  0.00000
## Balanced Accuracy  0.500000  0.53284  0.52289  0.50000

```

0.5

```
# K-means
set.seed(42)
kmeans_result <- kmeans(df[, c("Anxiety", "Depression", "Hours.per.day")], centers=3)
df$Cluster <- kmeans_result$cluster

#
library(ggplot2)
ggplot(df, aes(Anxiety, Depression, color=as.factor(Cluster))) + geom_point()
```

